GWV-xuk — Build Log & Endpoint Testing Guide

> Companion to `docs/BACKEND_GUIDE.md`. That file explains **how the backend works**. This file explains **what we built and changed**, **what every new file is for**, and **how to test every endpoint by hand** — written for a junior developer.

Table of contents

1. What this session changed (the story)
2. Every file we added or changed
3. Setup: env, migrations, run
4. How authentication works in practice
5. Three ways to test endpoints
6. The full endpoint list (with auth level)
7. Guided curl walkthrough by feature
8. Testing webhooks locally (M-Pesa + WhatsApp)
9. The `tests/` folder — what it is and why
10. Common errors and how to fix them
11. Junior tips + what to build next

1. What this session changed (the story)

We hardened and extended the backend in this order:

1. **RBAC** — added `require_admin`. Before this, admin routes only checked "are you logged in?", so *any* customer could delete inventory.
2. **IDOR fixes** — removed client-supplied `user_id` from `profiles`, `upload`, `wishlist`, `reviews`, and `orders/user/{id}`. The user now comes from the validated JWT only.
3. **CORS** — added `CORSMiddleware` so your React app won't be blocked.
4. **Checkout integrity** — server now recomputes prices/total from the DB, validates stock, and restores stock if payment fails/cancels.
5. **Merged M-Pesa webhooks** — one handler (`services/mpesa.py`) instead of two copies.
6. **`app/services/`** — extracted reusable business logic (inventory, mpesa, ai, whatsapp) out of the routers.
7. **Admin API** — a whole `app/routers/admin/` package under `/admin/*`, protected once at the group level.
8. **WhatsApp triage bot** — webhook + LLM qualification that scores leads as `hot` / `warm` / `cold`.
9. **Database migrations + RLS** — reproducible schema and Row Level Security policies.
10. **Project hygiene** — `requirements.txt`, `.env.example`, and this doc.
11. **Completed the missing endpoints** — cart update/remove/clear, wishlist remove, public `GET /categories`, auth `me` / `refresh` / `logout` / `forgot-password`, customer order cancel, and `GET /health`.
12. **Guest browsing model** — anyone can browse products/categories/reviews and track orders, but **cart, wishlist, checkout and paying now require login**. Checkout also records the owner (`user_id`), so `/orders/me` finally works.

13. **WhatsApp webhook signature verification** — Meta's `X-Hub-Signature-256` HMAC is checked against `WHATSAPP_APP_SECRET`.
14. **Automated tests (25)** — pytest suite covering stock idempotency, lead scoring, signature verification, and the security boundaries.
15. **Typed API client scaffold** — `frontend/` with a generated `openapi.json` → TypeScript types and a small auth-aware fetch client.

2. Every file we added or changed

New top-level / config files

File	Purpose
`requirements.txt`	Pinned dependencies (`pip install -r requirements.txt`)
`.env.example`	Documents every env var (safe to commit; no secrets)
`supabase/rls_policies.sql`	Row Level Security policies for existing tables
`docs/BACKEND_GUIDE.md`	How the backend works (beginner tour)
`docs/BUILD_LOG_AND_TESTING.md`	This file
`supabase/migrations/001_create_categories.sql`	`categories` table + `products.category_id`
`supabase/migrations/002_add_profiles_role.sql`	`profiles.role` column (`customer` / `staff` / `admin`)
`supabase/migrations/003_create_leads.sql`	`leads` , `whatsapp_messages` , `lead_events`
`supabase/migrations/004_rls_new_tables.sql`	`is_admin()` + RLS for new tables
`requirements-dev.txt`	Dev-only tools (`pytest`)
`pytest.ini`	Pytest config (`pythonpath` , `testpaths`)
`frontend/`	Typed API client scaffold (Phase 0) — see §12
`tests/conftest.py` , `tests/fakes.py`	Test setup + in-memory fake Supabase
`tests/test_pagination.py` , `test_inventory.py` , `test_whatsapp_helpers.py` , `test_security_boundaries.py`	25 automated tests

New application files

File	Purpose
`app/dependencies.py`	`get_current_user` + `require_admin` (RBAC)
`app/utils/pagination.py`	`page_range()` — safe page→row range
`app/services/__init__.py`	Marks `services` as a package
`app/services/inventory.py`	`restore_order_stock`, `release_stock_once` (idempotent)
`app/services/mpesa.py`	`process_stk_callback` — the one M-Pesa handler
`app/services/ai.py`	`chat_completion` — shared OpenRouter call
`app/services/whatsapp.py`	Triage bot: send, parse, score, notify owner
`app/routers/admin/__init__.py`	Parent admin router (applies `require_admin`)
`app/routers/admin/dashboard.py`	`GET /admin/dashboard/summary`
`app/routers/admin/orders.py`	List/get/status/cancel orders
`app/routers/admin/products.py`	Product list/update/delete + image management
`app/routers/admin/categories.py`	Category CRUD
`app/routers/admin/customers.py`	Customer list/detail
`app/routers/admin/inventory.py`	Low-stock + set stock
`app/routers/admin/delivery.py`	Delivery-location CRUD
`app/routers/admin/reviews.py`	Review moderation
`app/routers/admin/leads.py`	Lead management + manual reply
`app/routers/webhooks/__init__.py`	Marks `webhooks` as a package
`app/routers/webhooks/whatsapp.py`	Meta verify (`GET`) + receive (`POST`)
`app/routers/categories.py`	**Public** `GET /categories` for storefront navigation
`tests/simulate_mpesa_webhook.py`	**Rewritten** to match the real callback shape

Modified application files

File	Change
`app/main.py`	Added CORS, `admin_router`, `whatsapp_webhook_router`, `categories.router`, `GET /health`
`app/dependencies.py`	Added `require_admin`
`app/routers/products.py`	Admin guards on create/patch/delete; removed old `verify_admin`
`app/routers/orders.py`	Server-side pricing, safe stock, admin guards, merged webhook, **checkout now requires login + stores `user_id`, customer self-cancel**
`app/routers/payments.py`	Delegates to `services/mpesa.py`
`app/routers/auth.py`	Added `GET /me`, `POST /refresh`, `POST /logout`, `POST /forgot-password`
`app/routers/cart.py`	Added `PATCH /{item_id}`, `DELETE /{item_id}`, `DELETE /` (clear)
`app/routers/profiles.py`	`PATCH /profiles/me` (was `/update-profile/{user_id}`)
`app/routers/upload.py`	`POST /upload/profile-picture` (was `/{user_id}`)
`app/routers/wishlist.py`	Token-scoped (no `user_id` from client) + `DELETE /{product_id}`
`app/routers/reviews.py`	Token-scoped + fixed verified-buyer check
`app/routers/analytics.py`	Admin guard

3. Setup: env, migrations, run

3.1 Environment

```
cp .env.example .env      # then fill in real values
```

Minimum to boot: `SUPABASE_URL`, `SUPABASE_KEY`. For admin: `ADMIN_EMAILS` **or** a `profiles.role = 'admin'` row. For the bot: `WHATSAPP_*` + `OPENROUTER_API_KEY`. For payments: `DARAJA_*`.

3.2 Run the migrations

Open the Supabase dashboard → **SQL Editor**, and run, in order:

```
supabase/migrations/001_create_categories.sql
supabase/migrations/002_add_profiles_role.sql
supabase/migrations/003_create_leads.sql
supabase/migrations/004_rls_new_tables.sql
supabase/rls_policies.sql
```

> Only run `rls_policies.sql` **after** your backend uses the **service_role** key (otherwise your own API calls may get blocked by RLS).

3.3 Install and run

```
source venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Then open **http://127.0.0.1:8000/docs**.

4. How authentication works in practice

1. **Sign up:** `POST /auth/signup` with `{ "email": "...", "password": "..." }`.
2. **Log in:** `POST /auth/login` → you get `session.access_token` (the JWT).
3. **Send it on protected requests** as `Authorization: Bearer <access_token>`.
4. **Admin access:** either add your email to `ADMIN_EMAILS` in `.env` and restart, **or** run in Supabase SQL:

   ```
   update public.profiles set role = 'admin' where user_id = '<your-auth-uuid>';
   ```

   (Create the profile row first via `POST /profiles/me` if it doesn't exist.)

Save the token in a shell variable for the curl examples below:

```
TOKEN=$(curl -s -X POST http://127.0.0.1:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"you@example.com","password":"secret"}' \
  | python -c "import sys,json; print(json.load(sys.stdin)['session']['access_token'
echo "$TOKEN"
```

5. Three ways to test endpoints

Way 1 — Swagger UI (easiest, recommended for beginners)

Open `/docs`. Click any endpoint → **Try it out** → fill the body → **Execute**. For protected routes, click **Authorize** (top right) and paste your token.

Way 2 — curl (great for copy/paste and scripts)

See section 7.

Way 3 — VS Code REST Client / `.http` files

Create `tests/api.http` and write requests like:

```
@base = http://127.0.0.1:8000
@token = paste-token-here

GET {{base}}/products/
Authorization: Bearer {{token}}
```

Click "Send Request" above each block. Nice because it lives in your repo.

6. The full endpoint list (with auth level)

Legend: `public` = no login · `user` = any logged-in user · `admin` = admin role.

Auth & root

Method	Path	Auth	Notes
GET	`/`	public	Welcome message
GET	`/health`	public	Liveness probe
POST	`/auth/signup`	public	Create account
POST	`/auth/login`	public	Returns JWT session + refresh token
GET	`/auth/me`	user	Current user (id + email)
POST	`/auth/refresh`	public	Exchange refresh token for a new session
POST	`/auth/logout`	user	Revoke token (best effort)
POST	`/auth/forgot-password`	public	Send reset email (always returns success)

Products & categories

Method	Path	Auth	Notes
GET	`/products/`	public	Search/filter/sort
GET	`/products/{product_id}`	public	One product + images
GET	`/categories/`	public	Active categories (storefront nav)
POST	`/products/`	admin	Create (multipart form + images)
PATCH	`/products/{product_id}`	admin	Legacy update (form fields)
DELETE	`/products/{product_id}`	admin	Legacy delete

Delivery

Method	Path	Auth	Notes
GET	`/delivery/locations`	public	Active locations

Orders & payments

Method	Path	Auth	Notes
POST	`/orders/checkout`	user	**Login required**; server-priced; stores owner
POST	`/orders/{order_id}/cancel`	user	Cancel my own order + restore stock
GET	`/orders/track/{order_ref}`	public	Track by `GWV-xxxx`
GET	`/orders/me`	user	My orders
GET	`/orders/user/{user_id}`	admin	A user's orders
GET	`/orders/all`	admin	All orders (legacy)
PATCH	`/orders/status/{order_id}`	admin	Legacy status update
PATCH	`/orders/verify-payment/{order_id}`	admin	Manual receipt verify
POST	`/orders/status-result`	public	Legacy M-Pesa alias
POST	`/payments/callback`	public	Canonical M-Pesa callback

Cart, wishlist, reviews, profiles, upload, AI

Method	Path	Auth	Notes
POST	`/cart/add`	user	Add/increment
GET	`/cart/`	user	My cart
PATCH	`/cart/{item_id}`	user	Set line quantity (stock-checked)
DELETE	`/cart/{item_id}`	user	Remove one line
DELETE	`/cart/`	user	Clear the whole cart
POST	`/wishlist/`	user	Add
GET	`/wishlist/`	user	My wishlist
DELETE	`/wishlist/{product_id}`	user	Remove a product (easy toggle)
POST	`/reviews/`	user	Verified-buyer review
GET	`/reviews/{product_id}`	public	Reviews for a product
GET	`/profiles/me`	user	My profile
POST	`/profiles/me`	user	Create/upsert profile
PATCH	`/profiles/me`	user	Update profile
POST	`/upload/profile-picture`	user	Upload avatar
POST	`/ai/chat`	user	AI shopping assistant

Admin (`/admin/*` — all admin)

Method	Path
GET	`/admin/dashboard/summary`
GET	`/admin/orders` · `/admin/orders/{order_id}`
PATCH	`/admin/orders/{order_id}/status`
POST	`/admin/orders/{order_id}/cancel`
GET	`/admin/products` · `/admin/products/{product_id}`
PATCH	`/admin/products/{product_id}`
DELETE	`/admin/products/{product_id}`
POST	`/admin/products/{product_id}/images`
DELETE	`/admin/products/{product_id}/images/{image_id}`
GET/POST	`/admin/categories`
PATCH/ DELETE	`/admin/categories/{category_id}`
GET	`/admin/customers` · `/admin/customers/{user_id}`
GET	`/admin/inventory/low-stock`
PATCH	`/admin/inventory/{product_id}`
GET/POST	`/admin/delivery-locations`
PATCH/ DELETE	`/admin/delivery-locations/{location_id}`
GET	`/admin/reviews`
DELETE	`/admin/reviews/{review_id}`
GET	`/admin/leads` · `/admin/leads/stats` · `/admin/leads/{lead_id}` · `/admin/leads/{lead_id}/messages`
PATCH	`/admin/leads/{lead_id}`
POST	`/admin/leads/{lead_id}/reply` · `/admin/leads/{lead_id}/assign`

Webhooks

Method	Path	Auth	Notes
GET	`/webhooks/whatsapp`	public	Meta verification
POST	`/webhooks/whatsapp`	public	Incoming WhatsApp messages

Total: 59 paths / 75 operations.

Guest browsing model (important)

Capability	Guest (no account)	Logged-in user	Admin
Browse products & categories	✅	✅	✅
View reviews / delivery locations	✅	✅	✅
Track an order by reference	✅	✅	✅
Add to cart / wishlist	❌	✅	✅
Checkout & pay	❌	✅	✅
View own orders / profile	❌	✅	✅
Admin dashboard / management	❌	❌	✅

This is the "browse freely, sign up to buy" model. The lock is enforced by `Depends(get_current_user)` on cart/wishlist/checkout — a guest gets `401`.

7. Guided curl walkthrough by feature

> Assumes server on `http://127.0.0.1:8000` and `TOKEN` exported (section 4).

7.1 Auth

```
curl -X POST http://127.0.0.1:8000/auth/signup \
  -H "Content-Type: application/json" \
  -d '{"email":"junior@example.com","password":"SuperSecret123"}'

curl -X POST http://127.0.0.1:8000/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"junior@example.com","password":"SuperSecret123"}'

# who am I?
curl http://127.0.0.1:8000/auth/me -H "Authorization: Bearer $TOKEN"

# refresh an expired access token (pass the refresh_token from login)
curl -X POST http://127.0.0.1:8000/auth/refresh \
  -H "Content-Type: application/json" \
  -d '{"refresh_token":"<REFRESH_TOKEN>"}'

# log out
curl -X POST http://127.0.0.1:8000/auth/logout -H "Authorization: Bearer $TOKEN"

# forgot password (always returns success)
curl -X POST http://127.0.0.1:8000/auth/forgot-password \
  -H "Content-Type: application/json" \
  -d '{"email":"junior@example.com"}'
```

7.2 Public catalogue

```
curl "http://127.0.0.1:8000/products/?search=iphone&brand=Apple&sort_by=price&order=
curl http://127.0.0.1:8000/products/<PRODUCT_ID>
curl http://127.0.0.1:8000/categories/
curl http://127.0.0.1:8000/delivery/locations
```

7.3 Create a product (admin, multipart with images)

```
curl -X POST http://127.0.0.1:8000/products/ \
  -H "Authorization: Bearer $TOKEN" \
  -F "name=iPhone 13 128GB" \
  -F "price=65000" \
  -F "brand=Apple" \
  -F "description=Clean UK used" \
  -F "color=Midnight" \
  -F "images=@/absolute/path/to/photo1.jpg" \
  -F "images=@/absolute/path/to/photo2.jpg"
```

7.4 Admin: manage a product + images

```
# paginated list
curl "http://127.0.0.1:8000/admin/products?page=1&page_size=20" -H "Authorization: E

# partial JSON update
curl -X PATCH http://127.0.0.1:8000/admin/products/<PRODUCT_ID> \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"price":62000,"stock_quantity":10}'

# add images
curl -X POST http://127.0.0.1:8000/admin/products/<PRODUCT_ID>/images \
  -H "Authorization: Bearer $TOKEN" -F "files=@/path/photo.jpg"

# delete an image
curl -X DELETE http://127.0.0.1:8000/admin/products/<PRODUCT_ID>/images/<IMAGE_ID> \
  -H "Authorization: Bearer $TOKEN"

# delete product (also removes storage files)
curl -X DELETE http://127.0.0.1:8000/admin/products/<PRODUCT_ID> -H "Authorization:
```

7.5 Categories (admin)

```
curl -X POST http://127.0.0.1:8000/admin/categories \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"iPhones","description":"Apple devices"}'

curl http://127.0.0.1:8000/admin/categories -H "Authorization: Bearer $TOKEN"
```

7.6 Delivery locations (admin)

```
curl -X POST http://127.0.0.1:8000/admin/delivery-locations \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"name":"Nairobi CBD","cost":300,"active":true}'
# copy the returned "id" -> use as location_id at checkout
```

7.7 Cart (user)

```
curl -X POST http://127.0.0.1:8000/cart/add \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"product_id":"<PRODUCT_ID>","quantity":1}'

curl http://127.0.0.1:8000/cart/ -H "Authorization: Bearer $TOKEN"

# change quantity of a line (item_id comes from GET /cart/)
curl -X PATCH http://127.0.0.1:8000/cart/<ITEM_ID> \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"quantity":3}'

# remove one line
curl -X DELETE http://127.0.0.1:8000/cart/<ITEM_ID> -H "Authorization: Bearer $TOKEN

# clear the whole cart
curl -X DELETE http://127.0.0.1:8000/cart/ -H "Authorization: Bearer $TOKEN"

# wishlist remove (toggle off by product id)
curl -X DELETE http://127.0.0.1:8000/wishlist/<PRODUCT_ID> -H "Authorization: Bearer
```

7.8 Checkout (login required)

```
curl -X POST http://127.0.0.1:8000/orders/checkout \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "phone": "254700000000",
    "email": "buyer@example.com",
    "fulfillment": "delivery",
    "location_id": "<LOCATION_ID>",
    "delivery_address": "Kilimani, off Argwings",
    "payment_method": "M-Pesa",
    "items": [
      {"id": "<PRODUCT_ID>", "name": "iPhone 13", "price": 1, "qty": 1, "image": ""}
    ],
    "total": 1
  }'
```

> Notice `"price": 1, "total": 1` — **they are ignored**. The server charges the real DB price. This is the checkout-security fix in action.

The response contains `order_id` (the `GWV-xxxx` reference) and `checkout_request_id`.

7.9 Track an order (public) + cancel my order

```
curl http://127.0.0.1:8000/orders/track/GWV-abcd

# customer cancels their own order (must own it, must not be shipped/delivered)
curl -X POST http://127.0.0.1:8000/orders/<ORDER_UUID>/cancel -H "Authorization: Bea
```

7.10 Admin: dashboard, orders, cancel

```
curl http://127.0.0.1:8000/admin/dashboard/summary -H "Authorization: Bearer $TOKEN'

curl "http://127.0.0.1:8000/admin/orders?status=Pending%20PIN&page=1" -H "Authorizat

curl -X PATCH http://127.0.0.1:8000/admin/orders/<ORDER_UUID>/status \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"status":"Shipped"}'

# cancel + restore stock (idempotent)
curl -X POST http://127.0.0.1:8000/admin/orders/<ORDER_UUID>/cancel -H "Authorizatic
```

7.11 Customers & inventory (admin)

```
curl "http://127.0.0.1:8000/admin/customers?search=john" -H "Authorization: Bearer $
curl http://127.0.0.1:8000/admin/customers/<USER_UUID> -H "Authorization: Bearer $TC
curl "http://127.0.0.1:8000/admin/inventory/low-stock?threshold=5" -H "Authorizatior
curl -X PATCH http://127.0.0.1:8000/admin/inventory/<PRODUCT_ID> \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"stock_quantity":25}'
```

7.12 Reviews (public read, user write, admin moderate)

```
curl -X POST http://127.0.0.1:8000/reviews/ \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"product_id":"<PRODUCT_ID>","rating":5,"comment":"Fast delivery!"}'

curl http://127.0.0.1:8000/reviews/<PRODUCT_ID>
curl http://127.0.0.1:8000/admin/reviews -H "Authorization: Bearer $TOKEN"
curl -X DELETE http://127.0.0.1:8000/admin/reviews/<REVIEW_ID> -H "Authorization: Be
```

7.13 Profile & upload (user)

```
curl -X POST http://127.0.0.1:8000/profiles/me \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"full_name":"Junior Dev","phone_number":"254712345678"}'

curl http://127.0.0.1:8000/profiles/me -H "Authorization: Bearer $TOKEN"

curl -X POST http://127.0.0.1:8000/upload/profile-picture \
  -H "Authorization: Bearer $TOKEN" -F "file=@/path/avatar.jpg"
```

7.14 Leads (admin)

```
curl "http://127.0.0.1:8000/admin/leads?status=hot&page=1" -H "Authorization: Bearer
curl http://127.0.0.1:8000/admin/leads/stats -H "Authorization: Bearer $TOKEN"
curl http://127.0.0.1:8000/admin/leads/<LEAD_ID>/messages -H "Authorization: Bearer
curl -X PATCH http://127.0.0.1:8000/admin/leads/<LEAD_ID> \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"status":"hot","score":85}'
curl -X POST "http://127.0.0.1:8000/admin/leads/<LEAD_ID>/reply" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"message":"Hi! Our team will call you shortly."}'
```

8. Testing webhooks locally (M-Pesa + WhatsApp)

8.1 M-Pesa callback (no real money needed)

Use the rewritten simulator:

```
# success path (marks order Paid, sends receipt)
python tests/simulate_mpesa_webhook.py <CHECKOUT_REQUEST_ID>

# user cancelled (marks Cancelled and restores stock)
python tests/simulate_mpesa_webhook.py <CHECKOUT_REQUEST_ID> - --cancel
```

Get a `<CHECKOUT_REQUEST_ID>` from `GET /admin/orders` (status `Pending PIN`). After running, check the order status and the product's `stock_quantity`.

8.2 WhatsApp webhook

Verify (Meta does this once):

```
curl "http://127.0.0.1:8000/webhooks/whatsapp?hub.mode=subscribe&hub.verify_token=<V
# -> should print 12345
```

Send a fake inbound message:

```
curl -X POST http://127.0.0.1:8000/webhooks/whatsapp \
  -H "Content-Type: application/json" \
  -d '{
    "entry":[{"changes":[{"value":{
      "contacts":[{"profile":{"name":"Test Buyer"}}],
      "messages":[{"from":"254700000000","id":"wamid.TEST","type":"text","text":{"bo
    }}]}]
  }'
```

The endpoint returns immediately; processing happens in the background. Then check `/admin/leads`. If `WHATSAPP_*` isn't configured, the bot logs and skips the outbound send, but the lead + message are still stored.

> **Signature verification:** if `WHATSAPP_APP_SECRET` is set, the `POST` must carry a valid `X-Hub-Signature-256` header (Meta signs the raw body with HMAC-SHA256). Unsigned requests are rejected with `403`. When the secret is **not** set (local dev), the check is skipped and a warning is logged. Set it in production.

8.3 Exposing to the internet (for real Meta/Safaricom)

Use ngrok:

```
ngrok http 8000
# set DARAJA_CALLBACK_URL=https://<ngrok>/payments/callback
# set the WhatsApp webhook URL in Meta to https://<ngrok>/webhooks/whatsapp
```

9. The `tests/` folder — what it is and why

Why do we have tests at all?

Tests are **automated checks that your code still behaves correctly** after you change something. Without them, every edit risks silently breaking payments, stock, or security, and you only find out when a customer complains. Tests turn "it works on my machine, I think" into "the machine verified it for me."

There are two broad kinds:

- **Unit tests** — test one small function in isolation (e.g. `release_stock_once` logic). Fast, no network.
- **Integration / API tests** — spin up the app and call real endpoints, often with a test database. Slower, but prove the whole path works.

What's in the folder right now?

```
tests/
├── simulate_mpesa_webhook.py    # a manual simulator (not a pytest test)
├── conftest.py                  # pytest setup (dummy env + test client fixture)
├── fakes.py                     # in-memory fake Supabase for unit tests
├── test_pagination.py           # page_range() clamping
├── test_inventory.py            # stock restore is correct AND idempotent
├── test_whatsapp_helpers.py     # scoring, JSON parsing, signature verification
└── test_security_boundaries.py  # guests are blocked from protected routes
```

`simulate_mpesa_webhook.py` is a **helper script**. It pretends to be Safaricom so you can trigger the payment callback on demand. It's not run by a test runner — you run it yourself while the server is up. It builds the exact STK callback JSON, POSTs it to `/payments/callback`, and prints the response.

> Why it isn't a "real" test: a real test asserts a result automatically and can run in CI with no server. This script just sends a request and lets *you* look at the output. That's perfect for manual learning, less so for CI.

How to run it

```
# terminal 1
uvicorn app.main:app --reload

# terminal 2
python tests/simulate_mpesa_webhook.py <CHECKOUT_REQUEST_ID>
python tests/simulate_mpesa_webhook.py <CHECKOUT_REQUEST_ID> - --cancel
```

Real pytest tests (already written!)

There are now **25 automated tests**. Install dev tools once:

```
pip install -r requirements.txt -r requirements-dev.txt
```

Run them:

```
pytest              # or: venv/bin/pytest
```

What they cover (and why):

- `test_pagination.py` — the clamp math can't be abused to dump the whole table.
- `test_inventory.py` — stock is restored correctly **and only once** (`release_stock_once` is idempotent). This is the safety net for the payment bug.
- `test_whatsapp_helpers.py` — lead scoring thresholds, tolerant JSON parsing of LLM output, extracting messages from Meta's nested payload, and HMAC **signature verification**.
- `test_security_boundaries.py` — a guest with no token is refused on `/admin/*` , `/cart/` , `/wishlist/` , and `/orders/checkout` ; public routes stay public. This locks in the RBAC + IDOR + guest fixes.

They use an in-memory fake Supabase (`tests/fakes.py`) so they run in ~1 second with **no network and no real database**.

Why this matters for you: every security bug you fixed (RBAC, IDOR, checkout, stock) now has a test, so it can *never come back* without someone noticing.

How to USE the tests (hands-on cheat-sheet)

Run all tests (always do this before committing):

```
pytest                       # inside the activated venv
# or, without activating:
venv/bin/pytest
```

Run a single file / a single test, and see detail:

```
pytest tests/test_inventory.py                 # one file
pytest tests/test_inventory.py::test_release_stock_once_is_idempotent   # one test
pytest -v                                      # verbose: list every test name
pytest -k stock                                # only tests whose name matches "stoc
pytest -x                                      # stop at the first failure
pytest --lf                                    # re-run only the last failures
pytest -s                                      # show print() output (e.g. fake logs
```

Read the output: a `.` = pass, `F` = fail, `E` = error. Failures print the exact line and the `assert` that broke (e.g. `assert 5 == 3`), which is usually enough to see what changed.

How to READ the test files (so they teach you the app)

- `tests/conftest.py` — runs **before** your tests. It sets dummy env vars so importing the app doesn't need a real Supabase, then exposes a `client` fixture (a `TestClient`). Any test that takes `client` as an argument gets it injected automatically.
- `tests/fakes.py` — a tiny stand-in for the Supabase client. It understands the chained calls the app makes (`.table().select().eq().execute()`), storing rows in a plain dict. This is why the tests need no database.
- `tests/test_inventory.py` — read this first. It shows the whole pattern: create a `FakeSupabase` , put fake rows in `fake.store` , patch the module's `supabase` , call the function, then assert on the store.
- `tests/test_security_boundaries.py` — read this second. It shows how to test *authorization*: call an endpoint with no token and assert `401/403` .

How to ADD a new test (step by step)

Say you add a function `app/services/pricing.py`:

1. Create `tests/test_pricing.py`.
2. Import your function and the fake: `from tests.fakes import FakeSupabase`.
3. If the function uses Supabase, `monkeypatch.setattr(module, "supabase", fake)`.
4. Arrange → Act → Assert (AAA):

```
from tests.fakes import FakeSupabase
import app.services.pricing as pricing

def test_free_shipping_over_50000(monkeypatch):
    fake = FakeSupabase()
    monkeypatch.setattr(pricing, "supabase", fake)   # only if it uses the DB
    assert pricing.shipping_cost(total=60000) == 0    # Act + Assert
```

5. Run just your file: `pytest tests/test_pricing.py -v`.

For an endpoint test, use the `client` fixture and no token to assert it's protected:

```
def test_my_new_route_requires_auth(client):
    assert client.get("/my-new-route").status_code in (401, 403)
```

Rule of thumb: every bug you fix gets a test that fails before the fix and passes after. That's how a junior becomes trusted with production code.

10. Common errors and how to fix them

Symptom	Meaning	Fix
`401 Unauthorized`	Missing/invalid/expired JWT	Log in again; send `Authorization: Bearer <token>`
`403 Forbidden`	Logged in but not an admin	Add email to `ADMIN_EMAILS` or set `profiles.role='admin'`
`422 Unprocessable Entity`	Request body/params failed validation	Check `/docs` for the exact schema; fix types/required fields
`404 Not Found`	Wrong id/path, or row doesn't exist	Verify the id from a list endpoint
`500 Internal Server Error`	Server-side bug or missing env var	Read the server traceback; check `.env`
Browser CORS error	Origin not allowed	Add your site to `CORS_ORIGINS` and restart
`RuntimeError: Missing SUPABASE_URL or SUPABASE_KEY`	`.env` not found/filled	Create `.env` from `.env.example`
Webhook returns accepted but nothing happens	Processing is background/FK missing	Check server logs; ensure migration `003` ran

Debug tip: re-run the server in the foreground and watch the traceback in your terminal while you re-run the failing curl. The error message is usually literal.

11. Junior tips + what to build next

Habits that will make you good:

- Always read the endpoint's schema in `/docs` before calling it.
- Keep your `TOKEN` in a shell variable; don't paste it repeatedly.
- After any DB write, verify it with a `GET` — trust, but verify.
- Change one thing at a time, then test.
- Read error tracebacks **bottom-up**; the last line is the cause.

Build next (in order):

1. ✅ ~~`pytest` tests for checkout, inventory, RBAC/IDOR~~ — done (25 tests).
2. ✅ ~~Meta webhook signature verification~~ — done (`WHATSAPP_APP_SECRET`).
3. Auth completeness: login rate-limiting + a reset-password screen.
4. Make Supabase calls async (or `run_in_threadpool`) for load.
5. Structured logging + error monitoring (Sentry).
6. **Phase 0→1 of the frontend** (see `docs/FRONTEND_ROADMAP.md`).
7. The React admin dashboard consuming the `/admin/*` API.

12. Frontend Phase 0 — typed API client (already scaffolded)

```
frontend/
├── openapi.json              # exported from the running backend
├── README.md
├── scripts/generate-api.sh   # regenerate schema + TS types
└── packages/api/
    ├── package.json          # @gwx/api
    ├── tsconfig.json
    └── src/
        ├── schema.d.ts       # AUTO-GENERATED (59 paths) — do not edit
        ├── client.ts         # tiny fetch client (auth-aware, no deps)
        └── index.ts
```

Regenerate after backend changes:

```
uvicorn app.main:app --reload           # terminal 1
bash frontend/scripts/generate-api.sh   # terminal 2
```

Use it in React:

```
import { createApiClient, type paths } from "@gwx/api";

const api = createApiClient({
  baseUrl: import.meta.env.VITE_API_URL,
  getToken: () => localStorage.getItem("access_token"),
  onUnauthorized: async () => { /* refresh or redirect to /login */ },
});

const res = await api.get<{ data: Product[] }>("/products/", { search: "iphone" });
```

The generated `schema.d.ts` gives full editor autocomplete for every request and response. The client was type-checked (`tsc --strict`) and passes. Read `frontend/README.md` and `docs/FRONTEND_ROADMAP.md` to start Phase 1.

Keep this file updated whenever you add an endpoint or change a flow — future you (and your teammates) will thank present you.